Tabla de contenido

Ficheros de intercambio de datos 2
3.1. Introducción 2
3.2. JSON (JavaScript Object Notation) 4
¿Qué es JSON? 4
Características principales 4
JSON en Java 5
org.json 5
Gson 7
Jackson 8
¿Cuándo usar cada librería JSON en Java? 9
Anexo: Adaptadores personalizados en Gson 11
Anexo: Cómo maneja Jackson los mismos tipos 12
JSON con Python 13
JSON: Java vs Python 14
3.3 XML (Extensible Markup Language) 16
¿Qué es XML? 16
¿Qué es XPath? 17
XML con Java 19

Ficheros de intercambio de datos

3.1. Introducción

Hasta ahora hemos trabajado con ficheros binarios y con serialización de objetos en Java.

En esos casos, los datos se guardan en un formato propietario de Java, no legible fácilmente por humanos ni directamente interoperable con otros lenguajes o sistemas.

Para resolver este problema, en el desarrollo de aplicaciones modernas se utilizan formatos de intercambio de datos estandarizados.

Estos formatos cumplen tres características fundamentales:

- Son legibles→ pueden abrirse en un editor de texto y entenderse fácilmente.
- Son independientes del lenguaje de programación
- Son multiplataforma → permiten que diferentes sistemas intercambien información (aplicaciones de escritorio, servidores web, dispositivos móviles).

Formatos más habituales:

- **JSON (JavaScript Object Notation)**
    - Sintaxis ligera, basada en pares clave-valor y arrays.
    - Muy usado en APIs REST, microservicios y aplicaciones web modernas.
    - Compacto y rápido de procesar.
- **XML (eXtensible Markup Language)**
    - Lenguaje de marcado jerárquico con etiquetas.
    - Más verboso que JSON, pero ampliamente extendido en sistemas empresariales y en configuraciones.
    - Permite validación mediante DTD y XSD

Ambos formatos sustituyen, en la mayoría de los casos, a la serialización binaria tradicional, ya que permiten:

- Compartir datos entre aplicaciones hechas en lenguajes distintos.
- Guardar configuraciones y ficheros de datos de manera comprensible.
- Servir de soporte para la comunicación entre cliente y servidor.

Ejemplos:

Supongamos que tenemos un objeto Alumno con nombre y edad:

- En binario (serializado en Java) → el fichero contendrá bytes ilegibles.
- En JSON:

```
{ "nombre": "Ana", "edad": 20 }
```

- En XML:

```
<alumno>
   <nombre>Ana</nombre>
   <edad>20</edad>
</alumno>
```

Conclusión: Los ficheros de intercambio de datos aportan interoperabilidad y legibilidad, dos requisitos imprescindibles en el desarrollo actual de aplicaciones y servicios.

3.2. JSON (JavaScript Object Notation)

¿Qué es JSON?

- JSON (JavaScript Object Notation) es un **formato de intercambio de datos basado en texto**.
- Nació en el mundo web, pero hoy en día se utiliza en **cualquier aplicación** que necesite intercambiar información entre distintos sistemas.
- Es **ligero, fácil de leer para las personas** y **fácil de procesar por las máquinas**.

Características principales

- Representa los datos como **pares clave–valor**.
- Utiliza una **sintaxis muy sencilla**, heredada de JavaScript, pero independiente de él.
- Soporta **tipos básicos universales**:
    - Cadenas de texto ("Ana")
    - Números (25)
    - Booleanos (true, false)
    - Nulos (null)
    - Arrays ([1,2,3])
    - Objetos ({"nombre":"Ana", "edad":25})
- Es el formato más habitual en las **APIs REST** y en servicios web.
- Ejemplo: representa un objeto con varias propiedades: nombre, edad, notas (un array) y activo (booleano).

```
{
  "nombre": "Ana",
  "edad": 25,
  "notas": [8.5, 7.2, 9.1],
  "activo": true
}
```

JSON en Java

Java no trae soporte nativo completo para JSON en la librería estándar, por lo que se usan librerías externas. Las más utilizadas son:

- **org.json** → simple, incluida en muchos ejemplos introductorios.
- **Gson** (de Google) → ligera y sencilla.
- **Jackson** → muy extendida en aplicaciones empresariales, potente para proyectos grandes.

org.json

- org.json -> librería básica
- Muy usada en proyectos pequeños/medianos.
- Destaca porque convierte directamente entre **objetos Java ↔ JSON**
- Sintaxis muy clara y directa.
- Se ve claramente la correspondencia entre un **objeto Java** y su representación en **JSON**.
- Las clases más utilizadas: JSONObject, JSONArray, JSONTokener

CLASE JSONObject

- Representa un **objeto JSON** → pares clave–valor.
- Métodos más importantes:
  - put(String clave, Object valor) → añade un par clave–valor.
  - getString("clave"), getInt("clave"), getBoolean("clave") → obtiene el valor asociado a una clave.
  - optString("clave", valorPorDefecto) → igual que getString, pero no lanza excepción si la clave no existe.
  - toString() → devuelve el contenido en formato texto JSON.

```
JSONObject persona = new JSONObject();
persona.put("nombre", "Ana");
persona.put("edad", 25);

System.out.println(persona.toString());
// {"nombre":"Ana","edad":25}
```

CLASE JSONArray

- Representa un **array JSON** (lista de objetos o valores).
- Métodos más importantes:
    - put(Object valor) → añade un elemento al array.
    - getJSONObject(int index) → devuelve el objeto JSON en la posición indicada.
    - length() → devuelve el número de elementos.
    - isEmpty() → indica si está vacío.

```
JSONArray lista = new JSONArray();

JSONObject p1 = new JSONObject().put("nombre", "Ana").put("edad", 25);
JSONObject p2 = new JSONObject().put("nombre", "Luis").put("edad",
30);

lista.put(p1);
lista.put(p2);

System.out.println(lista.toString(4));
// [
//     {"nombre":"Ana","edad":25},
//     {"nombre":"Luis","edad":30}
// ]
```

CLASE JSONTokener

- Clase que interpreta un flujo de texto o fichero y lo convierte en tokens JSON.
- Es útil cuando leemos JSON desde ficheros o streams.
- Método clave:
    - nextValue() → devuelve el primer valor JSON válido encontrado (JSONObject, JSONArray o valor simple).

```
try (BufferedReader br =
Files.newBufferedReader(Path.of("personas.json"))) {
    JSONTokener tokener = new JSONTokener(br);
    Object raiz = tokener.nextValue();

    if (raiz instanceof JSONArray) {
        JSONArray array = (JSONArray) raiz;
        System.out.println("Total personas: " + array.length());
    }
}
```

Nota: revisar en el aula virtual los ejemplos code_examples de JSON. Hay distintos ficheros de ejemplo:

- DemoOrgJson.java: ejemplo clase JSONObject donde se crear un objeto JSON y se muestra por consola.
- GuardarJSONv1.java: ejemplo clase JSONObject donde crea un objeto JSON y lo escribe en un fichero dentro de un directorio "data" dentro del proyecto.
- LeerJSONv1.java: ejemplo lectura de un fichero JSON. Para ello, se emplea la clase JSONTokener.
- GuardarJSONv2.java: ejemplo de JSONArray donde se creará un array de objetos JSON y se guardará en un fichero dentro del directorio "data" del proyecto.
- LeerJSONv2.java: ejemplo de JSONTokener que lee de un fichero JSON

Los ejemplos anteriores están sobre un proyecto Maven, donde se ha configurado la dependencia de JSON en el fichero poom.xml. Si no creas un proyecto y le incluyes la dependencia, deberías descargar el .jar y agregarlo a las librerías de tu espacio de trabajo.

En el fichero poom.xml se agregaron las líneas dentro de <dependencias>:

```
  <dependency>
     <groupId>org.json</groupId>
     <artifactId>json</artifactId>
      <version>20240303</version>
    </dependency>
```

Gson

- **ibrería ligera y sencilla** desarrollada por Google.
- Muy utilizada en proyectos pequeños y medianos.
- La principal ventaja es que convierte automáticamente entre **objetos Java ↔ JSON** sin necesidad de construir manualmente JSONObject o JSONArray.
- Muy clara y directa: basta con indicar la clase del objeto que queremos convertir
- Se importa mediante Maven:

```
<dependency>
   <groupId>com.google.code.gson</groupId>
   <artifactId>gson</artifactId>
   <version>2.11.0</version>
</dependency>
```

Gson → clase principal para convertir objetos Java a JSON y viceversa.

- toJson(objeto) → convierte un objeto Java a cadena JSON.
- fromJson(cadenaJson, Clase.class) → convierte JSON a un objeto Java de la clase indicada.

Nota: revisar en el aula virtual los ejemplos code_examples de Gson. Hay distintos ficheros de ejemplo:

- DemoGson.java: crea un objeto Alumno que guarda en un JSON con la librería Gson.
- GuardarAlumnosGson.java. Crea una lista de objetos Alumno y los guarda en un fichero JSON dentro de un directorio del proyecto
- LeerAlumnosGson.java. Lee un fichero JSON y muestra por pantalla.

Jackson

- Librería muy extendida en proyectos grandes y aplicaciones empresariales.
- Muy completa: soporta JSON, XML, YAML...
- Se integra con frameworks como **Spring Boot**. **(MUY IMPORTANTE)**
- Principal ventaja: gran potencia para convertir JSON ↔ Objetos Java, con más opciones de configuración que Gson.
- Dependencia Maven:

```
<dependency>
   <groupId>com.fasterxml.jackson.core</groupId>
   <artifactId>jackson-databind</artifactId>
   <version>2.17.2</version>
</dependency>
```

Clases más utilizadas en Jackson:

ObjectMapper → clase principal. Convierte JSON ↔ objetos Java.

- writeValue(fichero, objeto) → guarda objeto como JSON en un fichero.
- readValue(fichero, Clase.class) → lee JSON y lo convierte en un objeto Java.

ObjectWriter → variante de ObjectMapper para configurar la salida (ejemplo: writerWithDefaultPrettyPrinter() para JSON con formato legible).

TypeReference<T> → necesario para deserializar colecciones genéricas (listas, mapas, etc.).

Nota: revisar en el aula virtual los ejemplos code_examples de Jackson. Hay distintos ficheros de ejemplo:

¿Cuándo usar cada librería JSON en Java?

- **org.json**
  Es la librería más sencilla. Permite crear objetos y arrays JSON de manera manual usando JSONObject y JSONArray. Es útil en **ejemplos didácticos**, pruebas rápidas o proyectos muy pequeños, pero no es recomendable en aplicaciones de mayor complejidad porque no ofrece automatismos ni opciones avanzadas.

- **Gson (Google)**
  Está pensada para trabajar con **objetos Java directamente**. Con pocas líneas de código convierte cualquier objeto Java en JSON (toJson) y viceversa (fromJson). Es muy clara y ligera, por lo que resulta ideal en **proyectos pequeños y medianos** donde se busca rapidez y legibilidad.

- **Jackson (FasterXML)**
  Es la más completa y potente. Además de JSON, soporta otros formatos como XML. Ofrece gran flexibilidad en la configuración, integración con frameworks (especialmente **Spring Boot**) y mejor rendimiento en proyectos de gran volumen de datos. Por ello, es la elección habitual en **aplicaciones empresariales y servicios web**.

Librería	Clases principales	Características	Cuándo usarla
org.json	JSONObject, JSONArray, JSONTokener	- Muy simple y ligera. - No necesita configuración complicada. - Útil para crear y recorrer estructuras JSON básicas.	Introducción a JSON, ejemplos didácticos, proyectos pequeños.
Gson (Google)	Gson, GsonBuilder	- Convierte automáticamente objetos Java ↔ JSON. - Sintaxis clara y compacta. - Fácil de usar con colecciones.	Proyectos pequeños y medianos donde se necesite rapidez y sencillez.
Jackson (FasterXML)	ObjectMapper, ObjectWriter, TypeReference	- Librería más potente y flexible. - Soporta JSON, XML, YAML. - Se integra con frameworks como Spring Boot.	Proyectos grandes, aplicaciones empresariales, APIs REST.

Anexo: Adaptadores personalizados en Gson

Gson convierte automáticamente los tipos de datos más comunes de Java (String, int, boolean, listas, etc.), pero **no reconoce tipos complejos** como LocalDate o LocalDateTime..

```
LocalDate fecha = LocalDate.of(2025, 10, 7);
String json = new Gson().toJson(fecha);
```

Esto fallaría o produciría algo extraño, porque Gson **no entiende** LocalDate.

Por eso debemos **enseñarle cómo** convertir (serializar) y leer (deserializar) un LocalDate. Para estos casos se crean **adaptadores personalizados**

¿Qué es un Adapter?

Un **adaptador** es una clase que enseña a Gson cómo:

- **Serializar** → pasar un objeto Java a formato JSON.
- **Deserializar** → reconstruir el objeto Java a partir del JSON.

Para ello, la clase implementa las interfaces:

Interfaz	Qué hace
JsonSerializer<T>	Define cómo convertir un objeto Java T en JSON.
JsonDeserializer<T>	Define cómo leer un JSON y convertirlo a un objeto T.

Ejemplo: adaptar LocalDate

```
public class GsonLocalDateAdapter
    implements JsonSerializer<LocalDate>, JsonDeserializer<LocalDate> {

    private static final DateTimeFormatter FMT =
DateTimeFormatter.ISO_LOCAL_DATE;

    @Override
    public JsonElement serialize(LocalDate src, Type typeOfSrc,
JsonSerializationContext ctx) {
        return new JsonPrimitive(FMT.format(src)); // "2025-10-07"
    }

    @Override
    public LocalDate deserialize(JsonElement json, Type typeOfT,
JsonDeserializationContext ctx)
            throws JsonParseException {
        return LocalDate.parse(json.getAsString(), FMT);
    }
}
```

Y registrar el adaptador:

```
Gson gson = new GsonBuilder()
        .setPrettyPrinting()
        .registerTypeAdapter(LocalDate.class, new GsonLocalDateAdapter())
        .create();
```

Ejemplo gson-adapters-demo del aula virtual: proyecto Maven completo con Gson que demuestra el uso de **adapters** para varios tipos "no triviales":

- LocalDate → "yyyy-MM-dd"
- LocalDateTime → "yyyy-MM-dd'T'HH:mm:ss"
- Path → representación como String con la ruta del sistema

Anexo: Cómo maneja Jackson los mismos tipos

Jackson **sí** reconoce esos tipos de fecha y hora, siempre que registremos el módulo adecuado:

```
ObjectMapper mapper = new ObjectMapper();
mapper.registerModule(new JavaTimeModule()); // soporte LocalDate,
LocalDateTime...
mapper.disable(SerializationFeature.WRITE_DATES_AS_TIMESTAMPS);
mapper.enable(SerializationFeature.INDENT_OUTPUT); // pretty printing
```

Resultado:

```
{
  "id": "P-1001",
  "fecha": "2025-10-07"
}
```

Módulos Jackson más útiles:

Módulo	Qué añade soporte
jackson-datatype-jsr310	Fechas y horas de Java 8 (LocalDate, Instant, etc.)
jackson-datatype-jdk8	Tipos Optional y clases de Java 8
jackson-module-parameter-names	Reconoce parámetros de constructores
jackson-datatype-hibernate	Entidades JPA/Hibernate
jackson-datatype-json-org	Compatibilidad con org.json

Estos módulos se registran con:

```
mapper.registerModule(new JavaTimeModule());
```

JSON con Python

- En Python, el manejo de JSON es más sencillo que en Java porque el lenguaje incluye soporte nativo en su **librería estándar** (json).
- Esto permite trabajar directamente con cadenas, diccionarios y listas, sin necesidad de instalar librerías externas.

Clases y funciones principales (json)

- **json.dumps(obj)** Convierte un objeto Python (diccionario, lista, etc.) en una **cadena JSON**.
  Ejemplo: json.dumps(diccionario)
- **json.dump(obj, fichero)** Igual que dumps, pero escribe directamente en un **fichero físico**.
  Ejemplo: json.dump(diccionario, f)
- **json.loads(cadena)** Convierte una **cadena JSON** en un objeto Python (diccionario o lista).
  Ejemplo: json.loads(cadena_json)
- **json.load(fichero)** Igual que loads, pero lee directamente desde un **fichero JSON**.
  Ejemplo: json.load(f)
- **Parámetro indent=4** Permite generar JSON con formato legible (pretty printing).

Nota: revisar ejemplos del aula virtual json-Python: 01_basico_memoria.py, 02_guardar_alumnos.py, 03_leer_alumnos.py. Por otro lado, está el fichero pathlib_example.py que muestra ejemplos de uso de esta librería que aparecen en los ficheros anteriores.

JSON: Java vs Python

Aspecto	Java	Python
Soporte nativo	No (librerías externas)	Sí (json estándar)
Librerías típicas	org.json (básica), **Gson** (sencilla), **Jackson** (potente; estándar en Spring)	**json** estándar; opcionales: pydantic, orjson
Objeto → JSON (memoria)	new Gson().toJson(obj) / new ObjectMapper().writeValueAsString(obj)	json.dumps(obj, indent=4)
JSON → Objeto (memoria)	new Gson().fromJson(str, Clase.class) / new ObjectMapper().readValue(str, Clase.class)	json.loads(str)
Fichero: escribir	gson.toJson(obj, writer) / mapper.writeValue(file, obj)	json.dump(obj, file, indent=4)
Fichero: leer	gson.fromJson(reader, Tipo) / mapper.readValue(file, Clase.class)	json.load(file)
Colecciones genéricas	Gson: TypeToken<List<Alumno>>(){} ; Jackson: new TypeReference<List<Alumno>>() {}	Listas y diccionarios nativos; tipado opcional con typing/dataclasses
Modelos de datos	Clases Java con campos, getters/setters y (recomendable) constructor vacío	Diccionarios y listas; opcional @dataclass para definir clases con campos

Aspecto	Java	Python
Pretty printing	Gson: GsonBuilder().setPrettyPrinting(); Jackson: writerWithDefaultPrettyPrinter()	indent=4
Tipos no JSON (fecha/decimal)	Jackson: serializers/deserializers; Gson: TypeAdapter personalizados	default= en dump(s) y object_hook en load(s)
Validación de datos	Jackson + anotaciones / Bean Validation	pydantic o validaciones manuales
Streaming / grandes volúmenes	Jackson Streaming API (JsonParser/JsonGenerator)	ijson, JSON Lines (NDJSON)
Integración frameworks	Excelente (Spring Boot usa Jackson por defecto)	Muy simple en scripts; frameworks como FastAPI/Flask
Rendimiento	Jackson suele ser más rápido que Gson	json suficiente; orjson/ujson para máximo rendimiento
Curva de aprendizaje	Mayor (mapeo a clases, genéricos)	Menor (estructuras nativas)
Cuándo elegir	Backends empresariales, APIs REST con Spring, contratos estrictos	Scripts, ETL ligeros, prototipos, data science; APIs rápidas con FastAPI

3.3 XML (Extensible Markup Language)

¿Qué es XML?

- Lenguaje de marcado **jerárquico** con etiquetas y atributos.
- **Legible**, multiplataforma y **validadle** (DTD/XSD).
- Muy usado en **configuración** y en sistemas empresariales.
- A diferencia de JSON, es **más verboso**, pero ofrece **validación formal** y **namespaces** (espacios de nombres)

Ejemplo:

```
<?xml version="1.0" encoding="UTF-8"?>
<alumnos xmlns="http://ejemplo.org/alumnos">
  <alumno id="A001">
    <nombre>Ana</nombre>
    <edad>20</edad>
  </alumno>
  <alumno id="A002">
    <nombre>Luis</nombre>
    <edad>22</edad>
  </alumno>
</alumnos>
```

Bien formado vs válido

- *Bien formado*: respeta reglas sintácticas
    - Hay una raíz única.
    - Las etiquetas se abren y cierran correctamente y están bien anidadas.
    - Los atributos están entre comillas, etc.
    - No comprueba contenido ni tipos, solo la “gramática” del documento
- **Válido (valid)**: además de estar bien formado, el XML **cumple un “contrato”** (un **esquema**) que define:
    - qué elementos pueden aparecer y en qué orden,
    - qué atributos son obligatorios u opcionales,
    - cuántas veces se repiten,
    - e incluso **tipos** (número, fecha, patrón…).

  Ese contrato puede escribirse en:
    - **DTD** (Document Type Definition) → más antiguo y menos expresivo.

- **XSD** (XML Schema Definition) → más moderno y **tipado**, soporta espacios de nombres, restricciones por patrón, rangos, etc.

Ejemplo DTD

```
<!--
  DTD: define la "forma" del XML.
  Aquí declaramos qué elementos existen, su orden, repeticiones y qué
atributos son obligatorios.
-->

<!-- La raíz del documento es <alumnos> y debe contener uno o más <alumno>
(el + significa 1..n) -->
<!ELEMENT alumnos (alumno+)>

<!-- Cada <alumno> debe contener EXACTAMENTE (en ese orden) <nombre> y <edad>
-->
<!ELEMENT alumno (nombre, edad)>

<!-- El elemento <alumno> TIENE un atributo 'id' de tipo texto (CDATA) y es
OBLIGATORIO (#REQUIRED) -->
<!ATTLIST alumno id CDATA #REQUIRED>

<!-- <nombre> contiene solo texto (#PCDATA = Parsed Character Data) -->
<!ELEMENT nombre (#PCDATA)>

<!-- <edad> contiene solo texto (#PCDATA). (La DTD no tipa números; si
quieres tipos usa XSD) -->
<!ELEMENT edad (#PCDATA)>
```

Ejemplo XML válido para DTD anterior:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE alumnos SYSTEM "alumnos.dtd">
<alumnos>
  <alumno id="A001">
    <nombre>Ana</nombre>
    <edad>20</edad>
  </alumno>
</alumnos>
```

Si pones <alumno> sin <edad>, **ya no sería válido** (porque el DTD exige nombre y edad).

¿Qué es XPath?

XPath (XML Path Language) es un **lenguaje de consultas** para **navegar y seleccionar partes de un documento XML**.
Piensa en XPath como "rutas" que apuntan a **nodos** (elementos, atributos, texto) dentro del árbol XML, igual que una ruta de carpetas apunta a un archivo.

¿Para qué sirve?

- **Localizar** elementos y atributos concretos (p. ej., todos los <alumno> con edad > 20).
- **Filtrar** por condiciones (predicados), posiciones o valores.
- **Obtener valores** de texto, contar nodos, etc.
- Se usa junto con DOM/SAX o librerías de *binding* para **consultar** un XML de forma declarativa.

Conceptos clave

- **Ruta**: describe el camino hasta los nodos (/, //, nombres de etiqueta).
- **Predicados**: filtros entre corchetes […] para condiciones/posiciones.
- **Atributos**: se referencian con @ (ej.: @id).
- **Funciones**: text(), count(), contains(), etc.
- **Contexto**: las rutas se evalúan desde un nodo de partida (normalmente la raíz).

Ejemplo. Suponiendo el siguiente XML:

```
<alumnos>
  <alumno id="A001"><nombre>Ana</nombre><edad>20</edad></alumno>
  <alumno id="A002"><nombre>Luis</nombre><edad>22</edad></alumno>
</alumnos>
```

- //alumno → todos los elementos <alumno>.
- //alumno[@id="A002"] → el <alumno> cuyo atributo id sea “A002”.
- //alumno[edad > 20]/nombre/text() → el **texto** de <nombre> donde <edad> > 20.
- count(//alumno) → **número** de alumnos.
- //alumno[1] → el **primer** <alumno> (en orden de documento).
- //alumno[nombre="Ana"] → alumnos cuyo <nombre> sea “Ana”.

Conclusión: **XPath es al XML lo que una ruta + un pequeño “SELECT” es a una base de datos**: una forma compacta de señalar **qué** parte del documento quieres y bajo **qué condiciones**.

Cuándo usar:

- Cuando tienes un **DOM** cargado y necesitas **buscar/filtrar** datos sin recorrer manualmente el árbol.
- Para construir **consultas expresivas y concisas** en validaciones, transformaciones o informes.

XML con Java

XML es un formato **jerárquico y legible** para intercambiar y almacenar datos. En Java podemos:

- **Leer** (parsear) XML.
- **Consultar** partes del documento (XPath).
- **Modificar y guardar** XML.
- **Validar** su estructura con **XSD/DTD**.
- **Mapear** entre clases Java ↔ XML (*binding*, JAXB/Jakarta).

Formas de trabajar con XML en Java

Enfoque	Idea principal	Cuándo usarlo	Pros	Contras
DOM (árbol)	Carga **todo** el XML en memoria como un árbol Document	Ficheros pequeños/medianos, necesitas **leer + modificar + guardar**	Sencillo de programar, permite editar	Consume memoria proporcional al tamaño
SAX (eventos)	**No** carga todo el XML: dispara **eventos** al leer secuencialmente	Ficheros grandes, lectura **solo** (sin modificar)	Muy eficiente en memoria	Código más “ceremonioso” si hay lógica compleja
Binding (JAXB/Jakarta)	Mapea **clases Java** ↔ XML (como Gson/Jackson pero para XML)	Cuando trabajas con **modelos de dominio**	Muy productivo, código limpio	Requiere anotar modelos/clases
XPath	Lenguaje de **consultas** sobre un DOM	Cuando debes **buscar/filtrar** nodos	Consultas expresivas	Necesita DOM ya cargado
Validación XSD/DTD	Comprobar que el XML **cumple** un esquema	Intercambio con terceros, contratos formales	Garantiza estructura y tipos	Configuración adicional

Nota dependencias: Para DOM y SAX **no necesitas** dependencias externas (van en la JDK). Para *binding* (Java 11+), añade:

```
<dependencies>
  <!-- API JAXB/Jakarta -->
  <dependency>
    <groupId>jakarta.xml.bind</groupId>
    <artifactId>jakarta.xml.bind-api</artifactId>
    <version>4.0.2</version>
  </dependency>
  <!-- Implementación de referencia -->
  <dependency>
    <groupId>org.glassfish.jaxb</groupId>
    <artifactId>jaxb-runtime</artifactId>
    <version>4.0.5</version>
  </dependency>
</dependencies>
```